Bloque 2: tramos, matrices y recursión

Programación: concepto, ejemplo y problema de juez

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

Los temas

Cómo va cada tema

Tres partes en el mismo orden: el concepto que pide el problema, un ejemplo pequeño con su traza en una tabla, y el problema de UVa, resuelto entre todos y escrito en Python y en C++. Cada problema termina con lo que cuesta su solución frente a los límites del enunciado.

Tema	Problema
El mejor tramo de un arreglo	UVa 507, *Jill Rides Again*
Rectángulos en una matriz	UVa 108, *Maximum Sum*
Recursión sobre un árbol	UVa 839, *Not so Mobile*

Compilar, correr y comparar

C++

```
g++ -std=c++17 -Wall -Wextra solucion.cpp -o solucion
./solucion < 1.in > salida.out
diff salida.out 1.out
```

Python

```
python3 solucion.py < 1.in > salida.out
diff salida.out 1.out
```

Contenido

1 El mejor tramo de un arreglo

2 Rectángulos en una matriz

3 Recursión sobre un árbol

4 Tarea del bloque

Contenido

1. El mejor tramo de un arreglo
   - Concepto
   - Ejemplo
   - Ejercicio: UVa 507
2. Rectángulos en una matriz
3. Recursión sobre un árbol
4. Tarea del bloque

¿Qué guarda el recorrido de UVa 11078?

El ciclo de `open-credit-system.py`

```
def mayor_diferencia(a):
    mayor = a[0]
    mejor = a[0] - a[1]
    for j in range(1, len(a)):
        if mayor - a[j] > mejor:
            mejor = mayor - a[j]
        if a[j] > mayor:
            mayor = a[j]
    return mejor
```

Pregunta

Al terminar la vuelta j, ¿qué es `mayor` y qué es `mejor`? Dígalo en palabras, sin mirar los números de la muestra.

¿Qué guarda el recorrido de UVa 11078?

Respuesta: lo que el ciclo mantiene

`mayor` es la nota más alta entre $a[0]$ y $a[j]$, y `mejor` es la mayor diferencia $a[i] - a[k]$ con $i < k \leq j$: la mejor pareja vista hasta j. La vuelta $j + 1$ solo necesita esos dos valores para quedar igual de bien, y al salir del ciclo `mejor` ya es la respuesta.

El mismo recorrido para el mejor tramo

El mejor tramo se halla con un recorrido de la misma forma: dos variables que se ponen al día en cada casilla y dicen algo de todo lo que ya pasó.

Tramos y su suma

Cuántos hay

Un tramo de v es un bloque de casillas seguidas, $v[i..j]$ con $i \leq j$. Con n casillas hay $n(n+1)/2$, del orden de n^2: con 4 casillas, diez.

La suma de uno en $O(1)$

Con $acum[0] = 0$ y $acum[k+1] = acum[k] + v[k]$, la suma de $v[i..j]$ es $acum[j+1] - acum[i]$. Los acumulados se preparan con un recorrido, una sola vez.

Tramos y su suma

Con $v = [4, -5, 4, -3]$

k	0	1	2	3	4
$v[k]$	4	-5	4	-3	
$acum[k]$	0	4	-1	3	0

$$v[1.,2] = acum[3] - acum[1] = 3 - 4 = -1.$$

El mejor tramo que termina en cada casilla

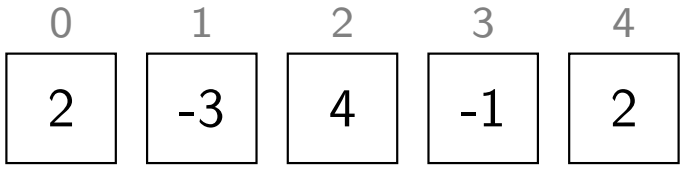
El concepto

En lugar de mirar todos los tramos, se recorre el arreglo una vez y en cada casilla se pregunta por el mejor tramo que termina ahí. Ese tramo es la casilla sola, o el mejor que terminaba en la casilla anterior más esta. Un recorrido, $\Theta(n)$, en vez de $n(n+1)/2$ tramos.

Por qué alcanza

Todo tramo termina en alguna casilla. Si para cada casilla se tiene el mejor que termina ahí, el mejor de todos está entre esos n valores.

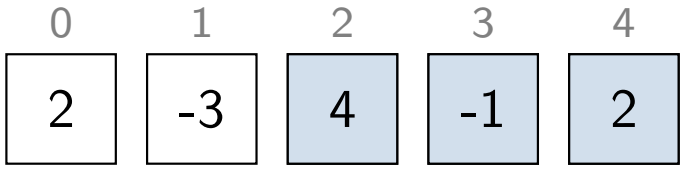
El tramo que más suma



A mano

Busque el tramo de casillas seguidas que más suma. Vale cualquier tramo, desde una casilla sola hasta el arreglo entero.

El tramo que más suma



Respuesta: 5

El tramo $v[2.,4]$, con $4 - 1 + 2 = 5$. Otros candidatos: $v[0.,0]$ suma 2, $v[0.,4]$ suma 4, $v[2.,2]$ suma 4. El -1 del medio se acepta porque lo que viene después lo paga.

La forma directa

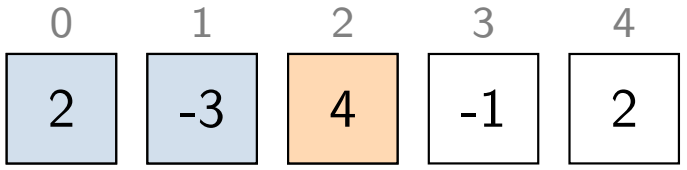
Python: `mejor_tramo.py`

```python
def mejor_tramo_acumulados(v):
    acum = [0] * (len(v) + 1)
    for k in range(len(v)):
        acum[k + 1] = acum[k] + v[k]
    mejor = v[0]
    for i in range(len(v)):
        for j in range(i, len(v)):
            suma = acum[j + 1] - acum[i]
            if suma > mejor:
                mejor = suma
    return mejor
```

¿Cuánto cuesta?

Los acumulados dan cada suma en $O(1)$, pero los tramos siguen siendo $n(n+1)/2$, del orden de n^2: con $n = 20\,000$, unos $2 \cdot 10^8$ por caso, y los casos vienen en tandas.

¿Conviene arrastrar lo que ya se lleva?



$v[0.,1] = -1$

Pregunta

El tramo que viene arrastrado desde la casilla 0 suma -1. Al llegar a la casilla 2, ¿conviene seguirlo o empezar un tramo nuevo ahí?

¿Conviene arrastrar lo que ya se lleva?

Respuesta

Empezar de nuevo. Un tramo que llega negativo solo resta: $-1 + 4$ da 3 y el que arranca en la casilla 2 da 4. La regla no depende de los números, solo del signo de lo que se lleva. Un recorrido, $\Theta(n)$, con dos variables: el tramo que termina en la casilla actual y el mejor visto.

Python: `mejor_tramo.py`

```
def mejor_tramo(v):
    mejor = v[0]
    actual = v[0]
    for k in range(1, len(v)):
        if actual < 0:
            actual = v[k]
        else:
            actual = actual + v[k]
        if actual > mejor:
            mejor = actual
    return mejor
```

La traza, paso a paso

Estado

`actual` es la mejor suma de un tramo que termina en k, y `mejor` la mejor de todas. Con $v = [2, -3, 4, -1, 2]$ las dos arrancan en $v[0] = 2$.

k	$v[k]$	¿arrastra?	`actual`	`mejor`
inicio			2	2
1	-3	sí, $2 \geq 0$	-1	2
2	4	?	?	?
3	-1	?	?	?
4	2	?	?	?

La traza, paso a paso

k	$v[k]$	¿arrastra?	`actual`	`mejor`
inicio			2	2
1	-3	sí, $2 \geq 0$	-1	2
2	4	no, $-1 < 0$	4	4
3	-1	sí, $4 \geq 0$	3	4
4	2	sí, $3 \geq 0$	5	5

Con todo negativo

En $[-4, -2, -7]$ el recorrido devuelve -2, la casilla menos mala. Arrancar `mejor` en 0 daría 0, que es la respuesta del tramo vacío: solo sirve si el enunciado acepta no tomar nada.

UVa 507: Jill Rides Again

Enunciado, en corto

Una ruta de bus tiene s paradas. Entre la parada i y la $i+1$ hay una calle con un valor n_i, distinto de cero, que dice cuánto le gusta a Jill. Ella baja en una parada, pedalea y vuelve a subir en otra. Hay b rutas, cada una con $2 \leq s \leq 20\,000$ paradas. Se pide el tramo de paradas $i < j$ con la mayor suma $n_i + \cdots + n_{j-1}$.

Desempates y salida

Si varios tramos tienen la suma máxima, gana el más largo (mayor $j - i$); si siguen empatados, el que empieza antes (menor i). Se imprime `The nicest part of route r is between stops i and j`, o `Route r has no nice parts` si la suma máxima no es positiva.

UVa 507: Jill Rides Again

Direcciones

Enunciado: `https://onlinejudge.org/external/5/507.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=448`

¿Entendimos el problema?

Muestra

```
3
3
-1 6
10
4 -5 4 -3 4
4 -4 4 -5
4
-2 -3 -4
```

En el archivo, un número por línea.

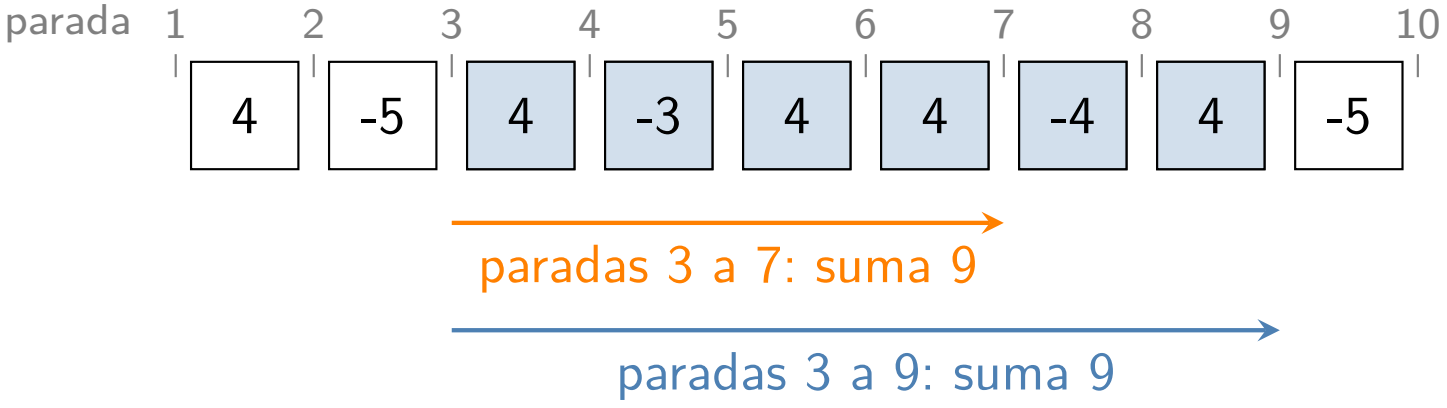
A mano

- Ruta 1: calles -1 y 6. Lo mejor es la segunda calle sola, de la parada 2 a la 3.
- Ruta 3: todas negativas. No hay tramo positivo: `Route 3 has no nice parts`.

Pregunta

En la ruta 2, el tramo de las paradas 3 a 7 suma $4 - 3 + 4 + 4 = 9$. ¿Por qué la respuesta es de la 3 a la 9?

¿Entendimos el problema?



Respuesta

Los dos suman 9 y el de 3 a 9 es más largo: las dos calles que agrega, -4 y 4, suman cero. Es el primer desempate.

Índices

`v[k]` une las paradas $k+1$ y $k+2$, así que el tramo de calles $v[i..j]$ va de la parada $i+1$ a la $j+2$.

Primera idea: todos los tramos

Python: `jill_acumulados.py`, con la suma de cada tramo en $O(1)$

```python
def mejor_tramo(v):
    acum = [0]
    for x in v:
        acum.append(acum[-1] + x)
    mejor = 0
    mi = 0
    mj = -1
    for i in range(len(v)):
        for j in range(i, len(v)):
            suma = acum[j + 1] - acum[i]
            if suma > mejor or (suma == mejor and j - i > mj - mi):
                mejor = suma
                mi = i
                mj = j
    return mejor, mi, mj
```

El desempate sale del orden: i crece afuera y la comparación de largo es estricta, así que entre dos tramos iguales en suma y en largo queda el de menor i.

Primera idea: todos los tramos

¿Cuánto cuesta?

Los tramos son $(s-1)s/2$, unos $2 \cdot 10^8$ con $s = 20\,000$. Sin acumulados, cada suma cuesta además hasta s pasos y el total pasa a $O(s^3)$.

Medido (una ruta; Python 3.14, Ryzen 7 7435HS)

s	todos los tramos	un recorrido
5 000	1,29 s	0,02 s
10 000	5,11 s	0,02 s
20 000	20,77 s	0,02 s

Contra los límites

Veinte segundos por una sola ruta de tamaño máximo, contra 3 segundos de límite para el archivo entero, que trae varias.

¿Sirve lo que se lleva sumado?

Pregunta

Se recorre la ruta llevando la suma del tramo que empezó en `inicio` y termina en la calle actual k. Si esa suma se vuelve negativa, ¿le conviene a algún tramo que siga después de k arrastrarla?

Con $v = [4, -5, 4, \ldots]$

Después de $4 - 5$ la suma es -1. Un tramo que empiece en la calle 2 vale lo mismo que con ese -1 delante, más uno.

¿Sirve lo que se lleva sumado?

Respuesta

No. Un prefijo negativo solo resta: la suma vuelve a 0 y el tramo empieza en $k + 1$. Es la idea de Kadane.

El recorrido, en palabras: $\Theta(s)$ por ruta

1. Sumar $v[k]$ a la suma actual.
2. Si la suma supera a la mejor, o la iguala con un tramo más largo, guardar el tramo $[inicio, k]$.
3. Si la suma quedó negativa, volver a 0 y poner $inicio = k + 1$.

¿Negativa o no positiva?

Se reinicia con $suma < 0$, no con $suma \leq 0$. Un prefijo que suma cero no resta, pero alarga el tramo, y el enunciado premia el tramo largo.

Traza con la ruta 2

k	$v[k]$	`suma`	`inicio`	`mejor`	tramo $[mi, mj]$
0	4	4	0	4	$[0, 0]$
1	-5	$-1 \to 0$	$0 \to 2$	4	$[0, 0]$
2	4	4	2	4	?
3	-3	?	?	?	?
4	4	?	?	?	?
5	4	?	?	?	?
6	-4	?	?	?	?
7	4	?	?	?	?
8	-5	?	?	?	?

Pregunta

En $k = 2$ la suma vuelve a 4, igual que la mejor. ¿Cambia el tramo guardado?

Traza con la ruta 2

k	$v[k]$	`suma`	`inicio`	`mejor`	tramo $[mi, mj]$
0	4	4	0	4	$[0, 0]$
1	-5	$-1 \to 0$	$0 \to 2$	4	$[0, 0]$
2	4	4	2	4	$[0, 0]$, mismo largo
3	-3	1	2	4	$[0, 0]$
4	4	5	2	5	$[2, 4]$
5	4	9	2	9	$[2, 5]$
6	-4	5	2	9	$[2, 5]$
7	4	9	2	9	$[2, 7]$, más largo
8	-5	4	2	9	$[2, 7]$

Respuesta

En $k = 2$ no: el tramo $[2, 2]$ tiene el mismo largo que $[0, 0]$ y empieza después.
En $k = 7$ sí: suma 9 otra vez, con más calles. Calles $[2, 7]$ son las paradas 3 a 9.

¿Qué mantiene el ciclo?

Pregunta

En la vuelta k, justo después de sumar $v[k]$ y comparar, ¿qué es `suma` y qué es `mejor`? Mire la traza: la columna de `suma` y la de `mejor` responden algo distinto.

¿Qué mantiene el ciclo?

Respuesta

`suma` es la suma del mejor tramo que termina en la calle k, y `mejor` la del mejor tramo visto hasta k, con sus extremos en $[mi, mj]$. La vuelta siguiente solo necesita esas dos cosas, y al salir del ciclo `mejor` ya es la respuesta de la ruta.

El mismo par que en UVa 11078

Allá eran `mayor` y `mejor`; aquí `suma` y `mejor`. Una variable resume lo que termina en la casilla actual y la otra todo lo visto.

Python: jill-rides-again.py

El recorrido

```
def mejor_tramo(v):
    # v[k] es la calle entre las paradas k + 1 y k + 2
    suma = 0
    inicio = 0
    mejor = 0
    mi = 0
    mj = -1
    for k in range(len(v)):
        suma = suma + v[k]
        if suma > mejor or (suma == mejor and k - inicio > mj - mi):
            mejor = suma
            mi = inicio
            mj = k
        if suma < 0:
            suma = 0
            inicio = k + 1
    return mejor, mi, mj
```

Python: jill-rides-again.py

La lectura y la salida

```
def main():
    datos = sys.stdin.read().split()
    b = int(datos[0])
    pos = 1
    for r in range(1, b + 1):
        s = int(datos[pos])
        v = []
        for k in range(s - 1):
            v.append(int(datos[pos + 1 + k]))
        pos = pos + s
        mejor, mi, mj = mejor_tramo(v)
        if mejor > 0:
            print("The nicest part of route %d is between stops %d and %d"
                  % (r, mi + 1, mj + 2))
        else:
            print("Route %d has no nice parts" % r)


main()
```

C++: jill-rides-again.cpp

El recorrido

```cpp
long long mejor_tramo(const vector<int>& v, int& mi, int& mj) {
    long long suma = 0;
    int inicio = 0;
    long long mejor = 0;
    mi = 0;
    mj = -1;
    for (int k = 0; k < (int) v.size(); k++) {
        suma = suma + v[k];
        if (suma > mejor || (suma == mejor && k - inicio > mj - mi)) {
            mejor = suma;
            mi = inicio;
            mj = k;
        }
        if (suma < 0) {
            suma = 0;
            inicio = k + 1;
        }
    }
    return mejor;
}
```

C++: jill-rides-again.cpp

```cpp
int main() {
    int b;
    scanf("%d", &b);
    for (int r = 1; r <= b; r++) {
        int s;
        scanf("%d", &s);
        vector<int> v(s - 1);
        for (int k = 0; k < s - 1; k++) {
            scanf("%d", &v[k]);
        }
        int mi, mj;
        long long mejor = mejor_tramo(v, mi, mj);
        if (mejor > 0) {
            printf("The nicest part of route %d is between stops %d and %d\n",
                   r, mi + 1, mj + 2);
        } else {
            printf("Route %d has no nice parts\n", r);
        }
    }
    return 0;
}
```

Trampas

Reiniciar con *suma* ≤ 0

Con $v = [1, -1, 5]$, los tramos $[0, 2]$ y $[2, 2]$ suman 5, y gana el largo: paradas 1 a 4. Si la suma se reinicia al llegar a cero, en $k = 1$ el tramo arranca de nuevo en la calle 2 y la respuesta sale de la parada 3 a la 4.

Comparar solo la suma

Con `suma > mejor` y nada más, la ruta 2 de la muestra da paradas 3 a 7.

La salida exacta

Paradas desde 1: la parada final es $mj + 2$. Cuando la mejor suma no es positiva va la otra frase, y la ruta se numera aunque no tenga tramo.

Variante para pensar

Ruta circular

Después de la última parada el bus vuelve a la primera, así que un tramo puede dar la vuelta: terminar en la calle 0 después de pasar por la última. ¿Cómo se halla la mejor suma con un recorrido?

Pista

Un tramo que da la vuelta deja por fuera un tramo del medio. Lo que queda es la suma total menos ese tramo: hay que quitar el de suma mínima, que también se halla con un recorrido.

¿Cuánto cuesta la solución?

De qué depende

De s, el número de paradas de una ruta, con $s \leq 20\,000$. El archivo no dice cuántas rutas trae, así que el costo se da por ruta.

Cuántas veces corre cada ciclo

La lectura y el ciclo de `mejor_tramo` corren $s - 1$ veces cada uno, con un trabajo fijo por vuelta: sumar, comparar y, a veces, guardar el tramo o reiniciar la suma.

Total, contra el límite

$\Theta(s)$ por ruta y $\Theta(s)$ de memoria para las calles. Con $s = 20\,000$ son unas $2 \cdot 10^4$ vueltas, frente a los $2 \cdot 10^8$ tramos de la primera idea.

El juez da 3 segundos para el archivo entero, y la primera idea gasta 20 en una sola ruta de tamaño máximo.

Contenido

1 El mejor tramo de un arreglo

2 Rectángulos en una matriz
- Concepto
- Ejemplo
- Ejercicio: UVa 108

3 Recursión sobre un árbol

4 Tarea del bloque

Una matriz y lo que cuesta

Cómo se guarda

En Python, una lista de listas: `m[i]` es la fila i y `m[i][j]` la celda. En C++, `vector<vector<int>> m(n, vector<int>(n));` crea n filas de n ceros.

Lo que cuesta

`m[i][j]` es $O(1)$. Recorrer una matriz de $f \times c$ es $\Theta(f \cdot c)$; sumar una fila es $\Theta(c)$ y una columna, $\Theta(f)$.

Cada fila por separado

En Python, `[[0] * n] * n` guarda n veces la misma fila, y escribir en una cambia todas. Las filas se crean una por una, con `append` dentro de un ciclo.

Acumulados por columna

Pregunta

$col[c]$ es la suma de la columna c entre las filas `arriba` y `abajo`. Si `abajo` baja una fila, ¿hay que sumar todo otra vez?

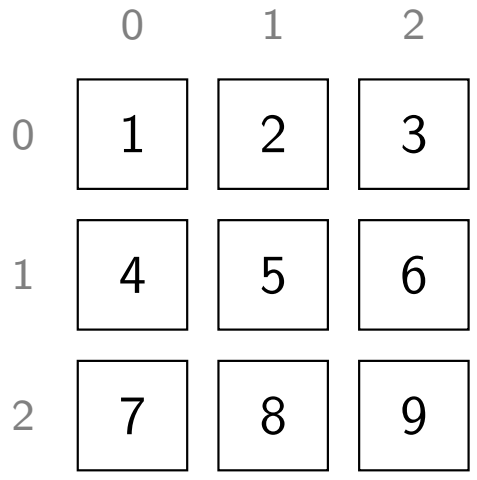
Respuesta

No: a cada columna se le suma la celda nueva, $col[c] \mathrel{+}= m[abajo][c]$. Cuesta $\Theta(n)$ por fila.

Con la matriz de la muestra de UVa 108 y $arriba = 1$

`abajo`	$col[0]$	$col[1]$	$col[2]$	$col[3]$
1	9	2	-6	2
2	5	3	-10	3
3	4	11	-10	1

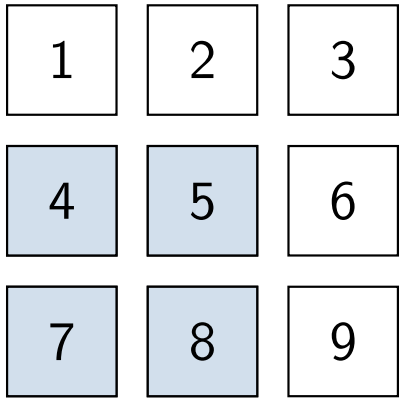
La suma de un rectángulo



A mano

¿Cuánto suma el rectángulo que va de la fila 1 a la 2 y de la columna 0 a la 1?

La suma de un rectángulo



Respuesta: 24

$4 + 5 + 7 + 8$. Recorrer las cuatro celdas es inmediato.

Pregunta

¿Y si hay que responder esa misma cuenta para miles de rectángulos de una matriz de 100×100?

La forma directa

Recorrer cada rectángulo

Dos ciclos anidados sobre las filas y las columnas del rectángulo: hasta $N \cdot M$ celdas por consulta.

Con muchas consultas

En una matriz de 100×100, un rectángulo grande cuesta 10^4 sumas. Mil consultas son 10^7, y cada consulta vuelve a sumar celdas que la anterior ya sumó.

Pregunta

En una dimensión, la suma de un tramo salía en $O(1)$ con los acumulados. ¿Qué sería el acumulado de una matriz?

La tabla de acumulados

Respuesta

Una tabla P con una fila y una columna más que la matriz: $P[i][j]$ acumula todo lo que queda arriba y a la izquierda de la celda (i, j). La fila 0 y la columna 0 quedan en ceros, que es el tramo vacío de una dimensión visto en dos.

Python: `acumulados2d.py`

```python
def tabla_acumulados(m):
    p = []
    for i in range(len(m) + 1):
        p.append([0] * (len(m[0]) + 1))
    for i in range(len(m)):
        for j in range(len(m[0])):
            p[i + 1][j + 1] = m[i][j] + p[i][j + 1] + p[i + 1][j] - p[i][j]
    return p
```

Llenar la tabla

Estado

Con la matriz 1 a 9, cada celda es $m[i][j]$ más la de arriba y la de la izquierda en P, menos la de la diagonal.

P	0	1	2	3
0	0	0	0	0
1	0	1	3	6
2	0	5	?	?
3	0	?	?	?

Para la casilla $P[2][2]$

$m[1][1] + P[1][2] + P[2][1] - P[1][1] = 5 + 3 + 5 - 1$. El rectángulo de arriba y el de la izquierda comparten una parte, que entra dos veces y por eso se descuenta.

Llenar la tabla

P	0	1	2	3
0	0	0	0	0
1	0	1	3	6
2	0	5	12	21
3	0	12	27	45

La esquina de abajo a la derecha

$P[3][3] = 45$ es la suma de toda la matriz, de 1 a 9.

Cuatro lecturas por rectángulo

Python: `acumulados2d.py`

```python
def suma_rectangulo(p, f1, c1, f2, c2):
    total = p[f2 + 1][c2 + 1] - p[f1][c2 + 1] - p[f2 + 1][c1] + p[f1][c1]
    return total
```

El rectángulo de hace un momento

Filas 1 a 2, columnas 0 a 1:
$P[3][2] - P[1][2] - P[3][0] + P[1][0] = 27 - 3 - 0 + 0 = 24.$

Las dos trampas

Si se olvida sumar de vuelta la esquina, el resultado se queda corto por la parte que se restó dos veces. Y los índices van corridos en uno: la celda (i, j) de la matriz vive en $P[i + 1][j + 1]$.

UVa 108: Maximum Sum

Enunciado, en corto

Una matriz de $N \times N$ enteros, positivos y negativos. Se pide la mayor suma de un subrectángulo: un bloque de filas y columnas seguidas, de 1×1 en adelante.

Límites

$N \leq 100$; cada valor en $[-127, 127]$. Tras N vienen los N^2 números fila por fila, separados por espacios y saltos de línea en cualquier reparto.

Direcciones

Enunciado: `https://onlinejudge.org/external/1/108.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=44`

¿Entendimos el problema?

Muestra, tal como viene

```
4
0 -2 -7 0 9 2 -6          2
-4 1 -4 1 -1
8 0 -2
```

0	-2	-7	0
9	2	-6	2
-4	1	-4	1
-1	8	0	-2

A mano

Sale un número: el bloque marcado, filas 1 a 3 y columnas 0 y 1, suma $9 + 2 - 4 + 1 - 1 + 8 = 15$.

Pregunta

La segunda línea trae 8 números y la última 3. ¿Qué dice eso de cómo leer?

Respuesta

Que se leen N^2 números por palabras; la celda (i, j) es el número $i \cdot N + j$.

Primera idea: todos los rectángulos

¿Cuántos hay?

Un rectángulo es un par de filas $f_1 \leq f_2$ y un par de columnas $c_1 \leq c_2$: $(N(N+1)/2)^2$. Con $N = 100$, $5050^2 \approx 2{,}6 \cdot 10^7$.

Sumando cada uno celda por celda

Cuatro ciclos para elegirlo y dos más para sumarlo: $O(N^6)$, del orden de 10^{10} sumas con $N = 100$. No pasa.

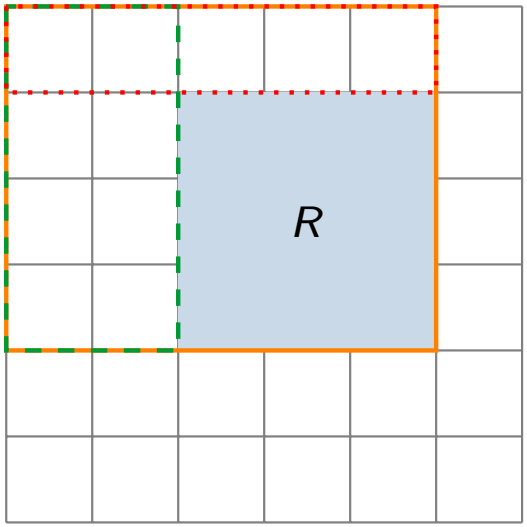
Pregunta

La suma de un tramo sale en $O(1)$ con acumulados. ¿Hay acumulados de un rectángulo?

Acumulados en dos dimensiones

Respuesta

Sí. $P[i][j]$ es la suma del rectángulo que va de la esquina $(0, 0)$ hasta la celda $(i-1, j-1)$, con P en ceros en la fila 0 y la columna 0. Un rectángulo cualquiera sale de cuatro esquinas:



$$R = P[f_2{+}1][c_2{+}1] - P[f_1][c_2{+}1] - P[f_2{+}1][c_1] + P[f_1][c_1]$$

La esquina de arriba a la izquierda se resta dos veces, por eso vuelve a sumarse.

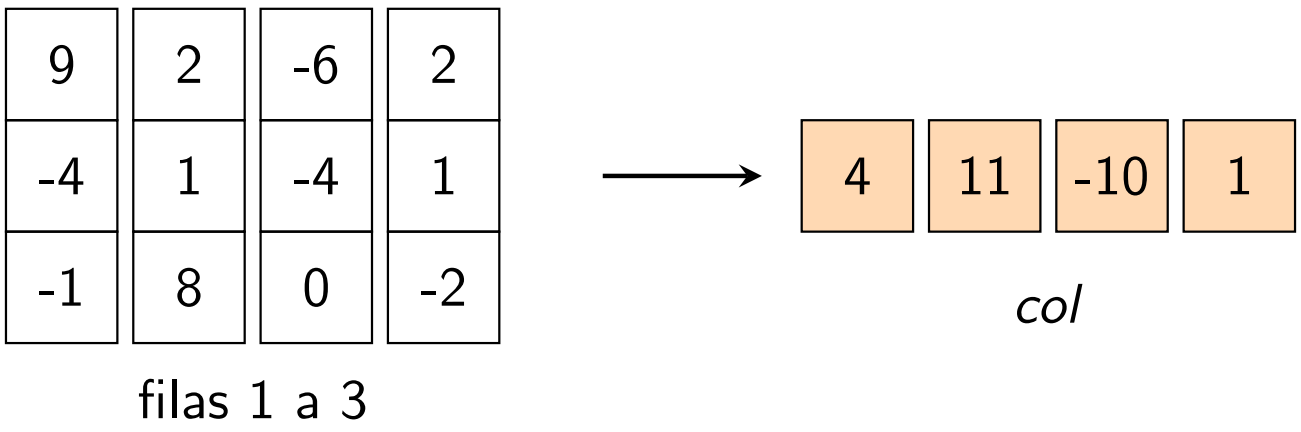
Costo

P se llena en $\Theta(N^2)$; cada rectángulo cuesta $O(1)$; todos, $O(N^4)$: unas $2{,}6 \cdot 10^7$ consultas, mil veces menos que antes.

¿Ya resolvimos esto?

Pregunta

Fije las filas `arriba` y `abajo`. Cada rectángulo entre ellas es un tramo de columnas. Si cada columna se cambia por su suma entre esas filas, ¿qué problema queda?



¿Ya resolvimos esto?

Respuesta

El del mejor tramo: el de suma máxima de *col*. Con $[4, 11, -10, 1]$ es $4 + 11 = 15$, el mismo rectángulo de la muestra.

El algoritmo

1. Para cada `arriba`, poner *col* en ceros.
2. Para cada `abajo` desde `arriba`: sumar la fila `abajo` a *col*, como en los acumulados por columna, y pasar *col* por el recorrido de Kadane.
3. Quedarse con el mayor resultado.

Traza con la muestra, *arriba* $= 1$

Kadane sobre *col*, sin desempates

Aquí solo importa la suma, y puede ser negativa: `mejor` arranca en *col*[0], no en 0.

`abajo`	*col*	mejor tramo de *col*
1	$[9, 2, -6, 2]$	$9 + 2 = 11$
2	$[5, 3, -10, 3]$	?
3	$[4, 11, -10, 1]$	?

Traza con la muestra, *arriba* $= 1$

Kadane sobre *col*, sin desempates

Aquí solo importa la suma, y puede ser negativa: `mejor` arranca en *col*[0], no en 0.

`abajo`	*col*	mejor tramo de *col*
1	$[9, 2, -6, 2]$	$9 + 2 = 11$
2	$[5, 3, -10, 3]$	$5 + 3 = 8$
3	$[4, 11, -10, 1]$	$4 + 11 = 15$

Resultado

El mejor de los diez pares de filas es 15, con `arriba` $= 1$ y `abajo` $= 3$.

Python: maximum-sum.py

El tramo de suma máxima

```
def mejor_tramo(v):
    mejor = v[0]
    suma = 0
    for x in v:
        suma = suma + x
        if suma > mejor:
            mejor = suma
        if suma < 0:
            suma = 0
    return mejor
```

Python: `maximum-sum.py`

Fijar dos filas

```
def mayor_rectangulo(m):
    n = len(m)
    mejor = m[0][0]
    for arriba in range(n):
        col = [0] * n
        for abajo in range(arriba, n):
            for c in range(n):
                col[c] = col[c] + m[abajo][c]
            s = mejor_tramo(col)
            if s > mejor:
                mejor = s
    return mejor
```

Python: `maximum-sum.py`

La lectura por palabras

```
def main():
    datos = sys.stdin.read().split()
    n = int(datos[0])
    m = []
    for i in range(n):
        fila = []
        for j in range(n):
            fila.append(int(datos[1 + i * n + j]))
        m.append(fila)
    print(mayor_rectangulo(m))


main()
```

C++: maximum-sum.cpp

El tramo de suma máxima

```
int mejor_tramo(const vector<int>& v) {
    int mejor = v[0];
    int suma = 0;
    for (int c = 0; c < (int) v.size(); c++) {
        suma = suma + v[c];
        if (suma > mejor) {
            mejor = suma;
        }
        if (suma < 0) {
            suma = 0;
        }
    }
    return mejor;
}
```

C++: maximum-sum.cpp

Fijar dos filas

```cpp
int mayor_rectangulo(const vector<vector<int>>& m) {
    int n = m.size();
    int mejor = m[0][0];
    for (int arriba = 0; arriba < n; arriba++) {
        vector<int> col(n, 0);
        for (int abajo = arriba; abajo < n; abajo++) {
            for (int c = 0; c < n; c++) {
                col[c] = col[c] + m[abajo][c];
            }
            int s = mejor_tramo(col);
            if (s > mejor) {
                mejor = s;
            }
        }
    }
    return mejor;
}
```

C++: maximum-sum.cpp

La lectura

```cpp
int main() {
    int n;
    scanf("%d", &n);
    vector<vector<int>> m(n, vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &m[i][j]);
        }
    }
    printf("%d\n", mayor_rectangulo(m));
    return 0;
}
```

¿Cabe en int?

La mayor suma posible es $127 \cdot 100^2 = 1\,270\,000$.

Trampas

Todo negativo

Con $N = 1$ y la celda -5, la respuesta es -5: el rectángulo tiene al menos una celda. Si `mejor` arranca en 0, sale 0.

Leer por líneas

La muestra misma reparte la matriz en líneas de 8, 5 y 3 números. Un programa que lee N números por línea falla con la muestra.

`col` sin reiniciar

col vuelve a ceros cada vez que cambia `arriba`; si no, arrastra filas de arriba que ya no están en el rectángulo.

Variante para pensar

Una matriz que no es cuadrada

La matriz tiene $f = 10$ filas y $c = 10\,000$ columnas. ¿Qué conviene fijar: dos filas o dos columnas?

Pista

Fijar dos filas cuesta $O(f^2 c)$ y fijar dos columnas, $O(c^2 f)$. Ponga los números.

¿Cuánto cuesta la solución?

De qué depende

Del lado N de la matriz, con $N \leq 100$: la entrada trae N^2 números.

Cuántas veces corre cada ciclo

`arriba` toma N valores y `abajo` a lo más N por cada uno: menos de N^2 pares de filas. Por cada par, N vueltas ponen al día *col* y otras N hace el recorrido de Kadane.

Total, contra el límite

$O(N^3)$, del orden de 10^6 pasos con $N = 100$, y $\Theta(N^2)$ de memoria. La tabla P daba $O(N^4)$, unas $2{,}6 \cdot 10^7$ consultas, y sumar celda por celda $O(N^6)$.

El juez da 3 segundos. Con una matriz aleatoria de 100×100 la solución tarda 0,06 s en Python 3.14 y menos de 0,01 s en C++ (Ryzen 5 3600).

Contenido

1 El mejor tramo de un arreglo

2 Rectángulos en una matriz

3 Recursión sobre un árbol
- Concepto
- Ejemplo
- Ejercicio: UVa 839

4 Tarea del bloque

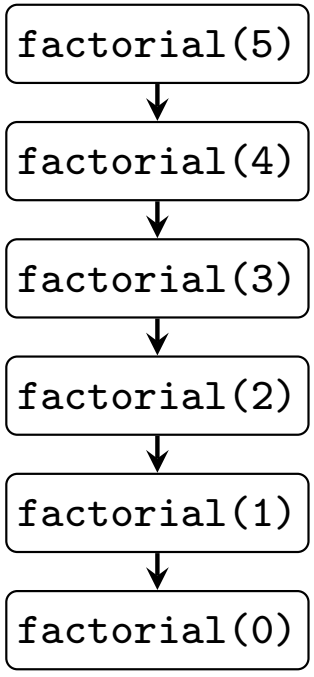
La pila de llamadas, en el depurador

De las estructuras y su costo

Cada llamada empuja un marco con sus parámetros y sus variables locales, y al terminar lo saca. La pila tiene un tamaño fijo, típicamente 8 MB en Linux, y Python corta a los 1000 niveles. En el depurador, `factorial(5)` detenido en el caso base:

```
(gdb) backtrace
#0  factorial (n=0) at factorial.cpp:8
#1  0x0000555555555168 in factorial (n=1) at factorial.cpp:10
#2  0x0000555555555168 in factorial (n=2) at factorial.cpp:10
#3  0x0000555555555168 in factorial (n=3) at factorial.cpp:10
#4  0x0000555555555168 in factorial (n=4) at factorial.cpp:10
#5  0x0000555555555168 in factorial (n=5) at factorial.cpp:10
#6  0x0000555555555193 in main () at factorial.cpp:17
```

Del `backtrace` al árbol de llamadas



El árbol

Cada llamada es un nodo y sus hijos son las llamadas que hace, de izquierda a derecha en el orden en que las hace. `factorial` hace una sola llamada, así que su árbol es una cadena de seis nodos.

La pila es un camino del árbol

En cada momento, la pila guarda el camino desde la raíz hasta la llamada que está corriendo: una llamada pendiente por nivel. El `backtrace` de arriba es ese camino leído de abajo hacia arriba.

Del `backtrace` al árbol de llamadas

Los dos casos

`n == 0` es el caso base: devuelve 1 sin llamar a nadie. El otro es el caso recursivo, `n * factorial(n - 1)`, que llama sobre una entrada más pequeña y así llega al base.

Lo que se lee en el árbol

- El número de nodos es el número de llamadas. Si cada llamada trabaja una cantidad constante fuera de sus llamadas, ese número es el costo.
- La altura del árbol, más uno, es el mayor número de marcos en la pila a la vez: lo que decide si la recursión cabe en 8 MB o en los 1000 niveles de Python.
- El orden en que se visitan los nodos, de arriba hacia abajo y de izquierda a derecha, es el orden en que se hacen las llamadas.

Dos llamadas por nivel

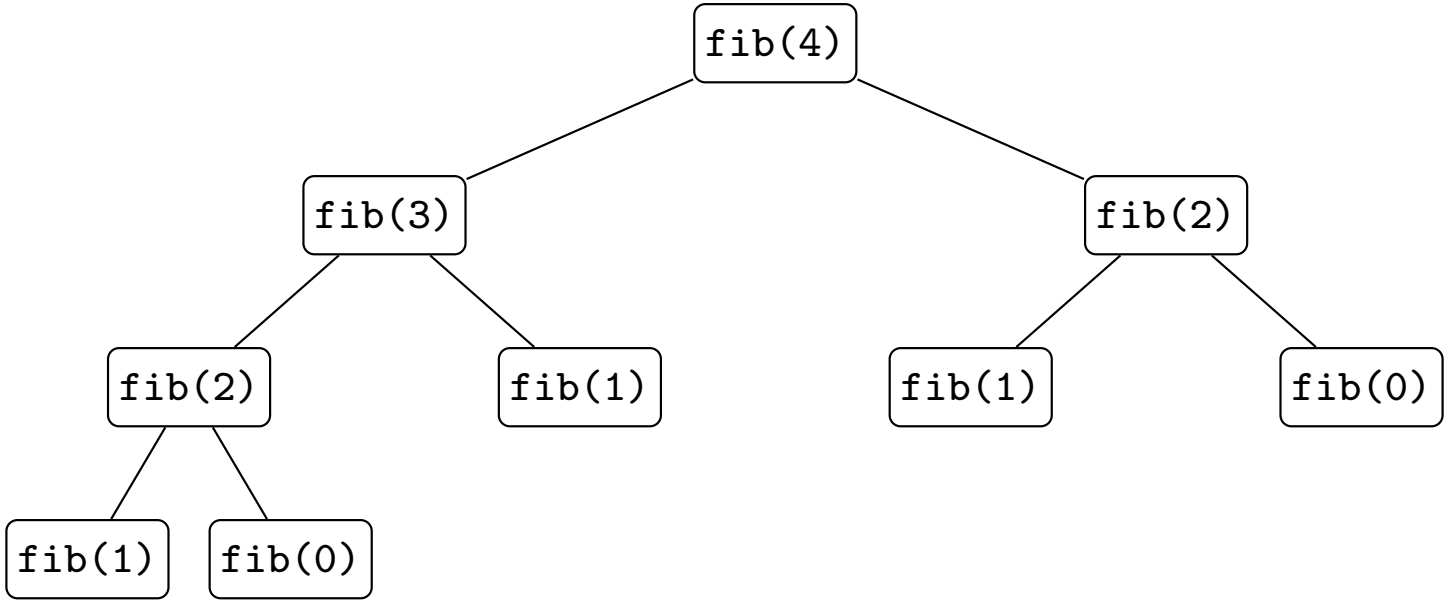
Python: `fib.py`

```python
def fib(n):
    if n <= 1:
        r = n
    else:
        r = fib(n - 1) + fib(n - 2)
    return r
```

Pregunta

Dibuje el árbol de llamadas de `fib(4)`. ¿Cuántas llamadas hay? ¿Cuántos marcos de `fib` hay en la pila como máximo?

El árbol de `fib(4)`



Respuesta

Nueve llamadas, y la función devuelve 3. El camino más largo baja por `fib(4)`, `fib(3)`, `fib(2)` y `fib(1)`: cuatro marcos a la vez. `fib(2)` se calcula dos veces, una en cada rama.

La pila mientras se recorre el árbol

Paso	Pila, del fondo al tope	Devuelve
1	`fib(4)`	
2	`fib(4)`, `fib(3)`	
3	`fib(4)`, `fib(3)`, `fib(2)`	
4	`fib(4)`, `fib(3)`, `fib(2)`, `fib(1)`	1
5	?	?
6	?	?
7	`fib(4)`, `fib(2)`	
8	?	?
9	?	?

Pregunta

En el paso 6, ¿qué llamada entra y qué tiene encima de ella?

La pila mientras se recorre el árbol

Paso	Pila, del fondo al tope	Devuelve
1	`fib(4)`	
2	`fib(4)`, `fib(3)`	
3	`fib(4)`, `fib(3)`, `fib(2)`	
4	`fib(4)`, `fib(3)`, `fib(2)`, `fib(1)`	1
5	`fib(4)`, `fib(3)`, `fib(2)`, `fib(0)`	0
6	`fib(4)`, `fib(3)`, `fib(1)`	1
7	`fib(4)`, `fib(2)`	
8	`fib(4)`, `fib(2)`, `fib(1)`	1
9	`fib(4)`, `fib(2)`, `fib(0)`	0

En el paso 6

`fib(2)` ya devolvió 1 y salió de la pila. Entra `fib(1)`, el segundo hijo de `fib(3)`, y no tiene nada encima: es un caso base.

UVa 839: Not so Mobile

Enunciado, en corto

Un móvil es una barra colgada de un hilo, con un objeto en cada extremo. Está en equilibrio si $W_l \times D_l = W_r \times D_r$, con los pesos W y las distancias D al punto de donde cuelga. En lugar de un objeto, de un extremo puede colgar otro móvil, y su peso es la suma de los pesos de sus objetos. Se pide decidir si todo el móvil está en equilibrio: cada una de sus barras.

Entrada

El número de casos, una línea en blanco, y entre caso y caso otra línea en blanco. Cada barra es una línea `Wl Dl Wr Dr`. Un peso 0 dice que de ese extremo cuelga un submóvil, descrito en las líneas que siguen: primero el izquierdo y después el derecho.

UVa 839: Not so Mobile

Salida

`YES` o `NO` por caso, con una línea en blanco entre dos casos.

Direcciones

Enunciado: `https://onlinejudge.org/external/8/839.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=780`

¿Entendimos el problema?

Muestra

```
1

0 2 0 4
0 3 0 1
1 1 1 1
2 4 4 2
1 6 3 2
```

Salida

```
YES
```

Pregunta

La primera barra tiene dos ceros. ¿Qué líneas describen el submóvil izquierdo y cuáles el derecho?

Pregunta

¿Cuánto pesa el submóvil izquierdo?

¿Entendimos el problema?

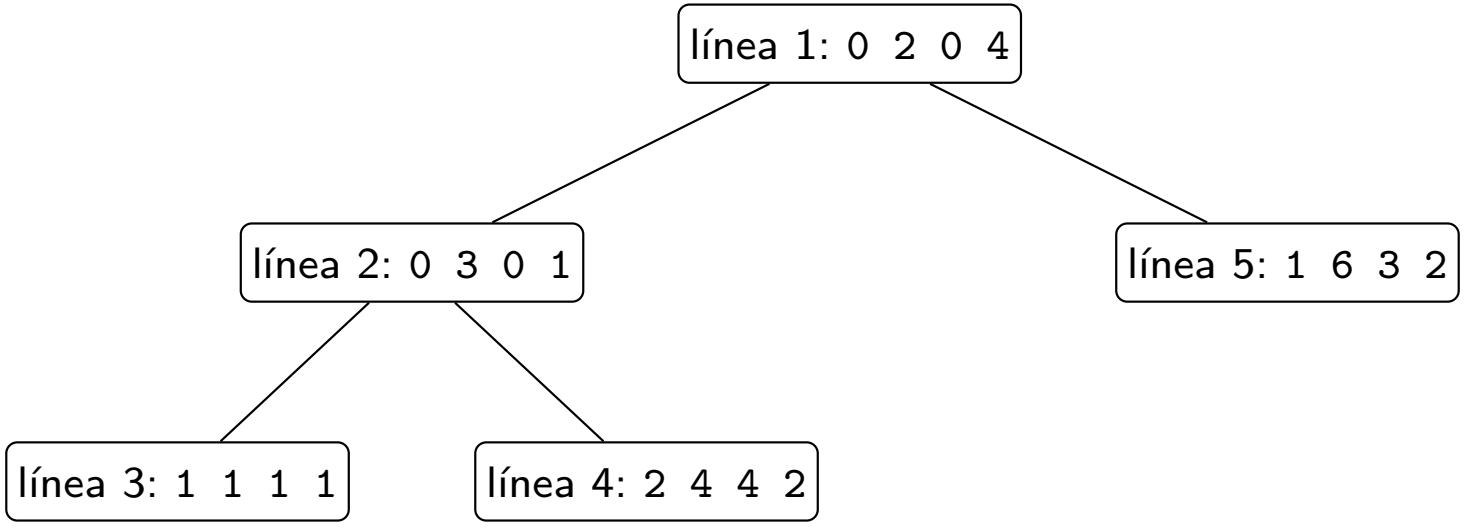
Respuesta

La línea 2, `0 3 0 1`, también tiene dos ceros: su submóvil izquierdo es la línea 3 y su derecho la línea 4. Con eso termina el submóvil izquierdo de la línea 1, y la línea 5 es el derecho. Las líneas no dicen dónde termina cada submóvil: hay que leerlas en orden y saber cuántas consume cada uno.

El peso

La línea 3 pesa $1 + 1 = 2$ y la 4, $2 + 4 = 6$; el submóvil izquierdo pesa 8. Su barra está en equilibrio porque $2 \cdot 3 = 6 \cdot 1$.

El móvil de la muestra



Las líneas y el árbol

Cada barra es un nodo, y cada peso 0 es un hijo. La entrada lista los nodos en el orden de arriba hacia abajo y de izquierda a derecha: la barra, después todo su lado izquierdo y después todo su lado derecho.

Primera idea: mirar cada barra sola

Pregunta

La línea 1 es `0 2 0 4`. ¿Se puede decidir si está en equilibrio con esos cuatro números?

Respuesta

No: $0 \cdot 2 = 0 \cdot 4$ siempre da verdadero, y los pesos de verdad están en las líneas de abajo. La barra necesita, de cada submóvil, dos cosas: cuánto pesa y si está en equilibrio por dentro.

La función

`movil` lee una barra. Por cada extremo con peso 0 se llama a sí misma, que lee todas las líneas de ese submóvil y devuelve su peso y si está en equilibrio. Con los dos pesos de verdad compara $W_l D_l$ con $W_r D_r$ y devuelve $W_l + W_r$ y el resultado.

Caso base y caso recursivo

Caso base

Un extremo con objeto: su peso viene en la línea y no hay nada que leer.

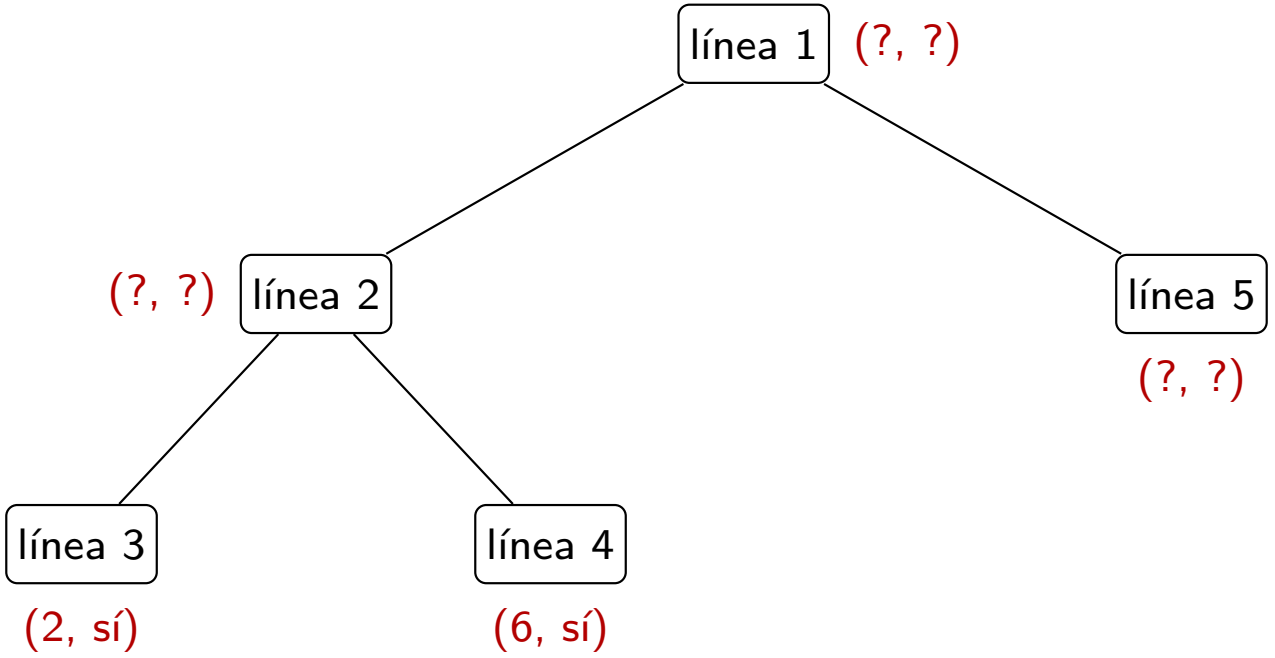
Caso recursivo

Un extremo con peso 0: una llamada que lee el submóvil completo. El submóvil tiene menos barras que el móvil, así que las llamadas terminan.

El árbol de llamadas es el móvil

Una llamada por barra, y cada llamada hace tantas llamadas como ceros tenga su línea: ninguna, una o dos.

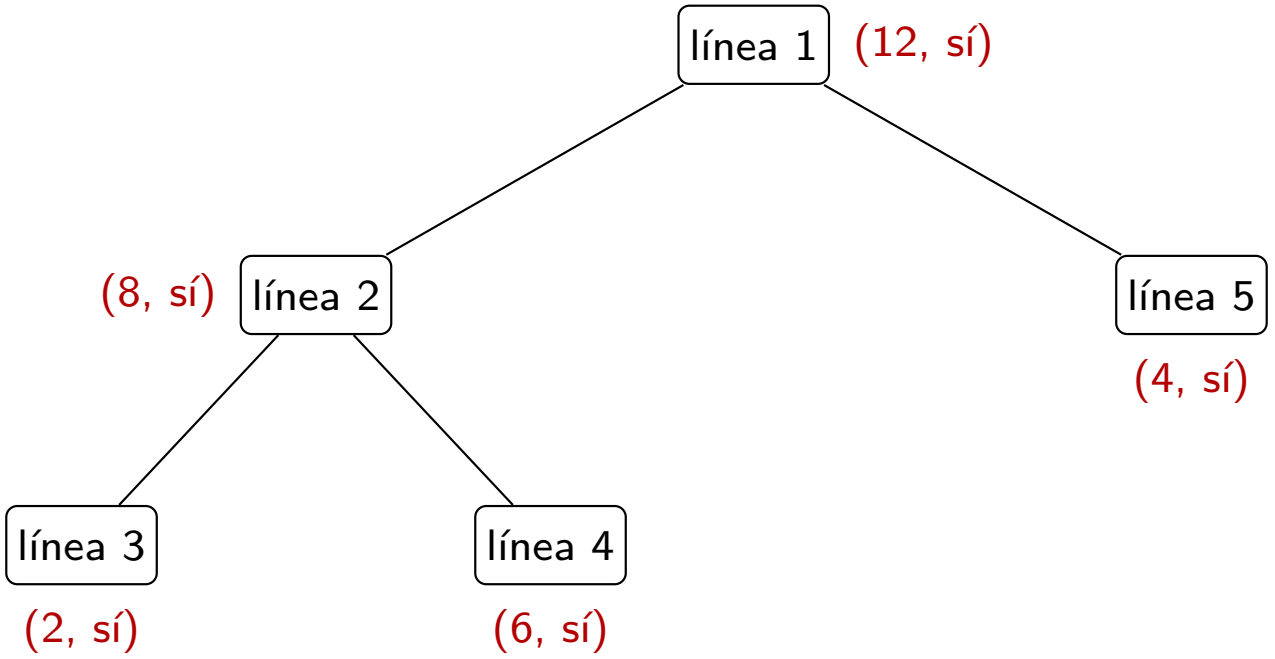
Lo que devuelve cada llamada



Pregunta

Las hojas ya devolvieron su par (peso, equilibrio). Complete los otros tres, de abajo hacia arriba.

Lo que devuelve cada llamada



Respuesta

Línea 2: $2 \cdot 3 = 6 \cdot 1$, pesa 8. Línea 5: $1 \cdot 6 = 3 \cdot 2$, pesa 4. Línea 1: con los pesos de verdad, $8 \cdot 2 = 4 \cdot 4$; todo el móvil pesa 12 y la salida es `YES`.

La pila mientras se lee la muestra

Paso	Pila, del fondo al tope	Lee	Devuelve
1	línea 1	`0 2 0 4`	
2	línea 1, línea 2	`0 3 0 1`	
3	línea 1, línea 2, línea 3	`1 1 1 1`	(2, sí)
4	?	?	?
5	línea 1, línea 2		?
6	?	?	?
7	línea 1		?

Pregunta

¿Cuál es el mayor número de marcos de `movil` en la pila? ¿Qué tiene que ver con el dibujo del móvil?

La pila mientras se lee la muestra

Paso	Pila, del fondo al tope	Lee	Devuelve
1	línea 1	0 2 0 4	
2	línea 1, línea 2	0 3 0 1	
3	línea 1, línea 2, línea 3	1 1 1 1	(2, sí)
4	línea 1, línea 2, línea 4	2 4 4 2	(6, sí)
5	línea 1, línea 2		(8, sí)
6	línea 1, línea 5	1 6 3 2	(4, sí)
7	línea 1		(12, sí)

Respuesta

Tres marcos, en los pasos 3 y 4: tantos como niveles tiene el móvil. Una línea se lee cuando su llamada entra a la pila, y una llamada sale solo cuando ya leyó todo lo que cuelga de ella.

Python: not-so-mobile.py

La función: devuelve el peso, el equilibrio y dónde sigue la entrada

```
def movil(datos, pos):
    # lee la barra que empieza en datos[pos] y todo lo que cuelga de ella;
    # devuelve el peso, si está en equilibrio y dónde sigue la entrada
    wl = int(datos[pos])
    dl = int(datos[pos + 1])
    wr = int(datos[pos + 2])
    dr = int(datos[pos + 3])
    pos = pos + 4
    equilibrio = True
    if wl == 0:
        wl, eq_izq, pos = movil(datos, pos)
        equilibrio = equilibrio and eq_izq
    if wr == 0:
        wr, eq_der, pos = movil(datos, pos)
        equilibrio = equilibrio and eq_der
    if wl * dl != wr * dr:
        equilibrio = False
    return wl + wr, equilibrio, pos
```

Python: not-so-mobile.py

Los casos

```
def main():
    sys.setrecursionlimit(10000)
    datos = sys.stdin.read().split()
    casos = int(datos[0])
    pos = 1
    for c in range(casos):
        peso, equilibrio, pos = movil(datos, pos)
        if c > 0:
            print("")
        if equilibrio:
            print("YES")
        else:
            print("NO")
```

sys.setrecursionlimit

Un móvil con más de mil niveles pasaría el límite de Python. El enunciado no da la altura máxima, y subir el límite a 10 000 cubre ese caso.

C++: not-so-mobile.cpp

La función: devuelve el peso y apaga equilibrio si una barra falla

```cpp
long long movil(bool& equilibrio) {
    long long wl, dl, wr, dr;
    scanf("%lld %lld %lld %lld", &wl, &dl, &wr, &dr);
    if (wl == 0) {
        wl = movil(equilibrio);
    }
    if (wr == 0) {
        wr = movil(equilibrio);
    }
    if (wl * dl != wr * dr) {
        equilibrio = false;
    }
    return wl + wr;
}
```

C++: not-so-mobile.cpp

Los casos

```
int main() {
    int casos;
    scanf("%d", &casos);
    for (int c = 0; c < casos; c++) {
        bool equilibrio = true;
        movil(equilibrio);
        if (c > 0) {
            printf("\n");
        }
        if (equilibrio) {
            printf("YES\n");
        } else {
            printf("NO\n");
        }
    }
    return 0;
}
```

Trampas

Dejar de leer cuando ya se sabe que es `NO`

Si la función no lee el submóvil derecho cuando el izquierdo ya falló, sus líneas quedan sin consumir y el caso siguiente arranca en mitad de este. Con los seis casos de `2.in`:

	1	2	3	4	5	6
esperado	NO	YES	NO	NO	NO	YES
sin leer el resto	NO	YES	YES	YES	NO	YES

Trampas

Las líneas en blanco

Se lee por palabras: los saltos de línea y las líneas vacías no cuentan.

La salida

La línea en blanco va entre dos casos, no después del último.

Pesos sin cota

El enunciado no acota los pesos ni las distancias, y la comparación multiplica un peso acumulado por una distancia: en C++ van en `long long`.

Revisar solo la barra de arriba

Un móvil cuya barra de arriba está en equilibrio y alguna de abajo no, se responde con `NO`: el caso 5 de `2.in` es así.

Variante para pensar

Cuántas barras fallan

En lugar de `YES` o `NO`, imprima cuántas barras del móvil no están en equilibrio. En el caso 5 de `2.in` son 2.

Pista

La función devuelve el peso y un contador en lugar del par. El contador de una barra es el de cada submóvil más 1 si ella misma falla.

¿Cuánto cuesta la solución?

De qué depende

Del número n de barras del móvil, que el enunciado no acota.

Cuántas llamadas hay

Una por barra, n por caso, sea cual sea la forma del móvil, y cada una hace un trabajo fijo: leer cuatro números, comparar dos productos y sumar dos pesos.

Total, contra el límite

$\Theta(n)$ por caso, igual que leer la entrada. La pila llega a la altura del móvil: unos $\log_2 n$ marcos si el árbol está lleno, n si es una cadena.

Lo que se puede acabar es la pila: una cadena de más de mil barras pasa el límite de Python, que la solución sube con `sys.setrecursionlimit`.

Contenido

1 El mejor tramo de un arreglo

2 Rectángulos en una matriz

3 Recursión sobre un árbol

4 Tarea del bloque

Para el bloque 3

Cómo se trabajan

Los tres se resuelven, se envían al juez y se revisan al comienzo del bloque siguiente. Van en orden de dificultad. Cada solución se entrega con su costo: de qué depende el tamaño de la entrada, cuántas veces corre cada ciclo o cuántas llamadas hay, y el total en O frente a los límites del enunciado.

Python en UVa

El juez corre una versión antigua de Python 3: la salida se arma con `print` y con `%`, como en las soluciones del bloque.

Base. UVa 100: The 3n + 1 problem

Direcciones

Enunciado: `https://onlinejudge.org/external/1/100.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=36`

Cómo atacarlo

Una función aplica la regla hasta llegar a 1 y cuenta los números que imprime; otra recorre el rango del menor al mayor y se queda con el máximo. La longitud cuenta el 1, así que la de 22 es 16. La entrada se lee hasta fin de archivo.

La trampa

i puede llegar mayor que j: el recorrido va del menor al mayor, pero la salida repite i y j en el orden en que llegaron. Los valores intermedios pasan de 10 000, así que no sirven como posición de un arreglo del tamaño de la entrada.

Intermedio. UVa 10189: Minesweeper

Direcciones

Enunciado: `https://onlinejudge.org/external/101/10189.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1130`

Cómo atacarlo

Por cada casilla libre se cuentan las minas de sus ocho vecinas, con los dos desplazamientos recorriendo $-1, 0, 1$ y la comprobación de borde antes del acceso. Las casillas con mina se copian tal cual. El primer número son las filas y el segundo las columnas.

La trampa

Está en la salida: la línea en blanco va entre campos y no después del último. Los bordes y las esquinas tienen menos de ocho vecinas, y un campo de 1×1 no tiene ninguna.

Exigente. UVa 620: Cellular Structure

Direcciones

Enunciado: `https://onlinejudge.org/external/6/620.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=561`

Cómo atacarlo

Una cadena sana es `A`, una sana seguida de `AB`, o `B` seguida de una sana y de `A`. La función recibe el rango [*ini*, *fin*]: el caso base es una letra, y la primera y la última letra deciden qué caso recursivo se intenta. Cada llamada hace a lo más una llamada, sobre un tramo dos letras más corto, sin copiar la cadena.

Exigente. UVa 620: Cellular Structure

La trampa

Lo de adentro tiene que ser una cadena sana y no vacía: `AB` y `BA` parecen formas válidas y son `MUTANT`. Contar letras no sirve, porque el interior se clasifica completo: `BAABA` es `MUTAGENIC` porque `AAB` es sana, y `BAAB` es `MUTANT` porque `BA` no lo es.

Para cerrar el bloque

La altura del móvil

Escriba en Python y en C++ una función recursiva que lea un móvil con el formato de UVa 839 y devuelva cuántos niveles tiene. Pruébela con la muestra, que tiene tres. ¿Cuántas llamadas hace con un móvil de n barras, y cuántos marcos llegan a estar en la pila a la vez?